

Задача 1 (10 точки)

Иван Иванов (студент от втори поток) има n приятелки. Всяка една от тях не знае за другите и Иван иска това да остане така.

На 14 февруари всяка негова приятелка иска да излезе на среща с него, като приятелка номер i му е казала, че иска да са заедно в интервала от l_i до r_i часа включително. Конкретно, ако една приятелка иска да започнат срещата си в час x , а друга му е казала, че иска тогава да приключат срещата си, това би означавало, че срещите се прекриват в момент x . Допълнително всяка от приятелките му отказва да променят по какъвто и да е начин часовете на срещата. Иван може да присъства на най-много една среща в даден момент.

Да се предложи алгоритъм с времева сложност $O(n \log n)$, който решава проблема DATE:

Вход: $L[1..n], R[1..n]$ – два масива от естествени числа, описващи интервалите от часове за срещи с приятелките.

Изход: `true`, ако Иван може да посети всички срещи,
`false` в противен случай.

Задача 2 (10 точки)

На лекция в СУ „Св. Климент Охридски“ присъстват неизвестен брой студенти. В почивката лекторът успява да зададе на n от тях следния въпрос: „Колко студенти тук са от твоята група?“. Всеки един от тях отговаря честно и кратко с естествено число. Лекторът се чуди „Колко най-малко студенти може да има на лекцията ми?“

Да се предложи алгоритъм с времева сложност $O(n \log n)$, който решава проблема ATTENDANCE:

Вход: $A[1..n]$ – масив от естествени числа, описващи отговорите на студентите, които лекторът е попитал.

Изход: m – естествено число, представляващо минималната бройка налични студенти, консистентна с дадените отговори.

Задача 3 (10 точки)

Вие сте лидер на армия от k войници и целите да нанесете максималното възможно поражение на Великата китайска стена. Тя може да бъде описана като разделена на n секции.

Всяка секция се характеризира със своята височина h_i и чупливост l_i . Това означава, че в началото секция i е висока h_i метра и когато изпратите войник да руши тази секцията, нейната височина ще бъде намалена с l_i метра, освен ако това означава секцията да остане с отрицателна височина – тогава височината ще стане 0 метра. Един войник ще руши най-много една секция и след това ще напусне войната. Може на една секция да бъдат изпратени няколко (включително няколко) войника.

Вие се интересувате да нанесете максимална щета на Великата китайска стена – това означава след вашата кампания сумарното разрушението, измерено в метри от стената, да бъде най-голямото възможно.

Да се предложи алгоритъм с времева сложност $O(n \log n)$, който решава проблема DESTRUCTION:

Вход: $H[1..n], L[1..n], k$ – два масива от естествени числа, описващи височината и чупливостта на всяка секция от стената и едно естествено число, описващо с колко на брой войници разполагаме.

Изход: d – естествено число, представляващо максималната сумарна бройка метри, които може да разрушите от стената по време на кампанията.

Решения, които предлагат алгоритъм с времева сложност $O(n+k \log n)$ на същия проблем ще се оценяват с най-много 5 точки.

Първо решение на задача 1 (Станислав Игов)

Наблюдение 1. Иван може да посети всички срещи тогава и само тогава, когато никои два интервала не се пресичат.

Наблюдение 2. Ако сортираме интервалите по краен момент $R[i]$, достатъчно е да проверим за всяко последователно i дали $R[i] \geq L[i+1]$. Ако интервал $i+1$ не се пресича с i -тия, то понеже $R[1] < R[2] < \dots < R[i]$, интервал $i+1$ не се пресича и с нито един от $1 \dots i$.

Algorithm 1: Date

Input: $L[1..n], R[1..n]$ - arrays of naturals
1 Sort(L, R) by R ascending (or L)
2 $flag \leftarrow \text{true}$
3 **for** $i \leftarrow 1$ **to** $n-1$ **do**
4 **if** $R[i] \geq L[i+1]$ **then**
5 $flag \leftarrow \text{false}$
6 **return** $flag$

Нека $flag_i$ е стойността на $flag$ преди i -тата итерация на цикъла на ред @3. Дефинираме:

$$I(i) \iff flag_i = \text{true} \iff \text{интервалите } 1 \dots i \text{ не се пресичат (по двойки)}.$$

Доказателство на коректност

- База $i = 1$. Имаме един интервал. Тогава няма пресичане, $flag_1 = \text{true}$. Следователно $I(1)$ е вярно.
- Поддръжка Допускаме, че $I(i)$ е вярно. Ще докажем $I(i+1)$. Разглеждаме два случая.
- Случай 1: $R[i] \geq L[i+1]$. Интервалите i и $i+1$ се пресичат. Тогава $flag \leftarrow \text{false}$ и $I(i+1)$ се запазва, тъй като интервалите $1 \dots i+1$ наистина се пресичат.
- Случай 2: $R[i] < L[i+1]$. Интервалите i и $i+1$ не се пресичат. Тъй като интервалите са сортирани по R , за всяко $j \leq i$ имаме $R[j] \leq R[i] < L[i+1]$, следователно интервал $i+1$ не се пресича с нито един от $1 \dots i$. Флагът не се променя и по индукционното допускане $I(i+1)$ се запазва: ако е имало пресичащи се в $1 \dots i$, те си остават; ако не е имало – продължава да няма.
- Терминация Цикълът завършва при $i = n$. По инварианта $I(n)$: $flag_n = \text{true} \iff$ интервалите $1 \dots n$ не се пресичат. Алгоритъмът връща $flag$, което е коректно.

Времева сложност

- @1: Сортиране – $\mathcal{O}(n \log n)$
- @3–@5: Линеен обход с константни операции – $\mathcal{O}(n)$

$$\mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n) \Rightarrow \text{Time}(\text{Date}) \in \mathcal{O}(n \log n).$$

Второ решение на задача 1 (Станислав Игов)

Дефинираме структура `Interval` с полета `start` и `end`, както и отношение за сравнение: $I_1 \leq I_2 \iff I_1.\text{end} \leq I_2.\text{end}$

Algorithm 2: Date

Input: $L[1..n], R[1..n]$ - arrays of naturals
1 $\text{Intervals}[1..n] \leftarrow \text{malloc}(n, \text{Interval})$
2 **for** $i \leftarrow 1$ **to** n **do**
3 $\text{Intervals}[i].\text{start} \leftarrow L[i]$
4 $\text{Intervals}[i].\text{end} \leftarrow R[i]$
5 MergeSort($\text{Intervals}[1..n]$)
6 **for** $i \leftarrow 1$ **to** $n-1$ **do**
7 **if** $\text{Intervals}[i].\text{end} \geq \text{Intervals}[i+1].\text{start}$ **then**
8 **return** `false`
9 **return** `true`

Нека i -тата итерация се отнася до цикъла на ред @6. Дефинираме:

$$I(i) \iff \text{интервалите } 1 \dots i \text{ не се пресичат по двойки.}$$

Тоест ако алгоритъмът не е върнал **false** преди i -тата итерация, то интервалите $1 \dots i$ са непресичащи се.

Доказателство

- База $i = 1$. Имаме един интервал – няма пресичане. $I(1)$ е вярно тривиално.
- Поддръжка Допускаме, че $I(i)$ е вярно, тоест интервалите $1 \dots i$ не се пресичат и алгоритъмът не е върнал **false**. Разглеждаме два случая.
- Случай 1: $\text{intervals}[i].\text{end} \geq \text{intervals}[i+1].\text{start}$: Интервалите i и $i+1$ се пресичат. Алгоритъмът връща **false** на ред @8, което е коректно.
- Случай 2: $\text{intervals}[i].\text{end} < \text{intervals}[i+1].\text{start}$: Тъй като масивът е сортиран по **end**, за всяко $j \leq i$ имаме $\text{intervals}[j].\text{end} \leq \text{intervals}[i].\text{end} < \text{intervals}[i+1].\text{start}$, следователно интервал $i+1$ не се пресича с нито един от $1 \dots i$. Заедно с индукционното допускане, че $1 \dots i$ не се пресичат помежду си, получаваме, че $1 \dots i+1$ също не се пресичат. Следователно $I(i+1)$ е вярно.
- Терминация Ако цикълът завърши без да върне **false**, то по инварианта $I(n)$ интервалите $1 \dots n$ не се пресичат по двойки. Алгоритъмът връща **true** на ред @9, което е коректно.

Следователно алгоритъмът **Date** е коректен.

Времева сложност

- @1: Заделяне на памет – $\mathcal{O}(n)$
- @2–@4: Запълване на масива – $\mathcal{O}(n)$
- @5: Сортиране – $\mathcal{O}(n \log n)$
- @6–@8: Линейно обхождане с константни операции – $\mathcal{O}(n)$

$$\mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n) \Rightarrow \text{Time}(\text{Date}) \in \mathcal{O}(n \log n).$$

Трето решение на задача 1 (Кристина Димитрова)

Algorithm 3: Date

Input: $L[1..n], R[1..n]$ – arrays of naturals

```
1 HeapSort( $L[1..n]$ ) // ascending;
2 HeapSort( $R[1..n]$ ) // ascending;
3 for  $i \leftarrow 1$  to  $n - 1$  do
4   if  $R[i] \geq L[i + 1]$  then
5     return false;
6 return true;
```

Времева сложност: Сортирането на масивите L и R отнема по $\mathcal{O}(n \log n)$. Цикълът извършва точно $n - 1$ проверки, което отнема $\mathcal{O}(n) \Rightarrow$ Общо $\mathcal{O}(n \log n)$.

Коректност на алгоритъма Коректността използва идея много подобна на задачата за “балансираните” скоби.

За да докажем коректността, трябва да покажем, че срещите на Иван се **препокриват тогава и само тогава**, когато съществува индекс i , за който $R_{\text{sorted}}[i] \geq L_{\text{sorted}}[i + 1]$.

Доказателство чрез броене на активните интервали: Иван не може да присъства на всички срещи точно тогава, когато съществува момент от време t , в който поне две срещи се препокриват.

Нека в произволен момент t дефинираме:

- p – броят на срещите, които вече са *започнали* (т.е. $L[k] \leq t$).
- q – броят на срещите, които вече са *приключили* преди този момент (т.е. $R[k] < t$, тъй като по условие краят x и началото x се водят за препокриване, считаме срещата за приключила само строго след нейния край).

Броят на активните срещи в момент t е точно $p - q$.

1. Ако има препокриване $\Rightarrow$ алгоритъмът ще върне **false**

Ако допуснем, че има застъпване в момент t , тогава активните срещи са поне две:

$$p - q \geq 2 \implies q \leq p - 2$$

Нека изберем индекс $i = q + 1$. Тъй като $q \leq p - 2$, следва че $i + 1 \leq p$.

- Тъй като точно q на брой срещи са приключили преди t , то $(q + 1)$ -вият най-малък край на среща трябва да е по-голям или равен на t . Следователно, $R_{sorted}[i] \geq t$.
- От друга страна, тъй като поне p на брой срещи са започнали до момент t , а $i + 1 \leq p$, то $(i + 1)$ -вото най-малко начало на среща също трябва да е по-малко или равно на t . Следователно, $L_{sorted}[i + 1] \leq t$.

Комбинирайки тези две наблюдения, получаваме:

$$R_{sorted}[i] \geq t \geq L_{sorted}[i + 1] \implies R_{sorted}[i] \geq L_{sorted}[i + 1]$$

Това е точно условието, при което нашият алгоритъм спира и връща **false**.

2. Ако няма препокриване $\Rightarrow$ алгоритъмът ще върне **true**

Ако нито една от срещите не се препокрива, те могат да бъдат подредени хронологично в една линия:

$$l_{k_1} \leq r_{k_1} < l_{k_2} \leq r_{k_2} < \dots < l_{k_n} \leq r_{k_n}$$

Тъй като масивите се сортират, L_{sorted} ще представлява точно подредбата на l_{k_i} , а R_{sorted} – подредбата на r_{k_i} . В този случай, край на i -тата среща ($R_{sorted}[i]$) винаги ще бъде строго преди началото на следващата $(i + 1)$ -ва среща ($L_{sorted}[i + 1]$). Тоест винаги ще е изпълнено $R_{sorted}[i] < L_{sorted}[i + 1]$ и алгоритъмът коректно ще премине през целия масив, връщайки накрая **true**.

Оценяване на задача 1 4 точки са заделени за правилен псевдокод, 2 от които за сортиране. Доказателството носи допълнителни 4 точки, 2 от които са за правилно описание кога два интервала се застъпват чрез неравенства. Финалните две точки са за завършване и времева сложност на алгоритъма.

Решение на задача 2 (Симеон Дойчинов)

За да намерим минималния брой студенти, ще използваме следните твърдения:

Наблюдение 1: Ако в масива имаме две различни стойности a_1 и a_2 , то тези отговори със сигурност са дадени от студенти от различни групи.

Наблюдение 2: Ако имаме s стойности в масива, които са равни на a_1 , то минималният брой групи, които могат да се образуват са $\lceil \frac{s}{a_1} \rceil$.

Доказателство: Ако допуснем, че можем да имаме по-малко на брой групи, то ще трябва някоя от тях да съдържа повече от a_1 студенти, което не е възможно. До това заключение може да се достигне, ако просто допуснем обратното - тоест групите са по-малко от $\lceil \frac{s}{a_1} \rceil$, но във всяка има не повече от a_1 студенти. Така ще достигнем до заключението, че общият брой на дадените отговори a_1 е по-малък от s , което е противоречие.

Сега използвайки тези две твърдения, ще сортираме входния масив низходящо и за всеки подмасив, който се състои от s на брой стойности a_i ще отбелязваме $\lceil \frac{s}{a_i} \rceil$ на брой групи.

Algorithm 4: Attendance

```
Input:  $A[1..n]$  – arrays of naturals
1 MergeSort( $A[1..n]$ )
2 CurrentValue  $\leftarrow A[1]$ 
3 TotalStudents  $\leftarrow 0$ 
4 StudentsCount  $\leftarrow 1$ 
5 for  $i \leftarrow 2$  to  $n - 1$  do
6   if  $A[i] \neq \text{CurrentValue}$  then
7     StudentsToAdd  $\leftarrow \text{CurrentValue} \times \lceil \frac{\text{TotalStudents}}{\text{CurrentValue}} \rceil$ 
8     TotalStudents  $\leftarrow \text{TotalStudents} + \text{StudentsToAdd}$ 
9     StudentsCount  $\leftarrow 1$ 
10  else
11    StudentsCount  $\leftarrow \text{StudentsCount} + 1$ 
12 return TotalStudents
```

Доказателство Нека след сортирането в масива имаме стойностите $a_1 \geq a_2 \geq a_3 \cdots \geq a_n$. С G ще означим решението, което се получава, когато от s на брой стойности a_i образуваме $\lceil \frac{s}{a_i} \rceil$ групи от по a_i студенти и една от $s \bmod a_i$ студенти. Ще докажем, че то е оптимално.

Нека разгледаме едно оптимално решение OPT, което използва този подход до стойността a_{i-1} , тоест до тази стойност в масива решенията OPT и G са еднакви. Конструираме OPT' от OPT, като за стойността a_i ще вземем $\lceil \frac{s}{a_i} \rceil$ групи от a_i студенти и една от $s \bmod a_i$. Използвайки твърдение 2, стигаме до заключението, че $|\text{OPT}'| \leq |\text{OPT}|$, тоест OPT' е оптимално и до стойността a_i е еднакво с решението G. По този начин след краен брой итерации можем да заключим, че G е оптимално.

Времева сложност

- @1: Сортиране - $O(n \log n)$
- @6–@15: Линеен обход с константни операции - $O(n)$

$$O(n \log n) + O(n) = O(n \log n) \Rightarrow \text{Time}(\text{MinStudents}) \in O(n \log n)$$

Оценяване на задача 2 2 точки времева сложност, 3 точки за правилен псевдокод, 5 точки за доказателство на коректност и наблюденията, които водят до нея.

Решение на задача 3 (Иван Лупов)

Задачата е много близка до изучаваната на упражнения FRACTIONALKNAPSACK. За да я сведем напълно ще изискаме единствено наблюдение да отделим “остатъците” от секциите – конкретно, ако имаме стена с височина 17 и чупливост 3, ще казваме, че тя има *остатък* 2, защото след многократно пращане на войници до нея, в някакъв момент стената ще остане с височина 2 и последният войник пратен на нея ще намали стената с 2 метра, а не с 3.

Така формално, една входна двойка (h_i, l_i) се разделя на две двойки $(h_i - (h_i \bmod l_i), l_i)$ (двойка-стена) и $(h_i \bmod l_i, h_i \bmod l_i)$ (двойка-остатък). От тук можем да приложим стандартното решение на FRACTIONALKNAPSACK – сортираме двойките относно l_i (печалбата за единица войник) в намаляващ ред и итеративно за всяка една от тях изчисляваме колко войника можем да пратим на нея като $\min\{k, \frac{h_i}{l_i}\}$. Важно е да отбележим тук, че $l_i > (h_i \bmod l_i)$ и съответно всяка двойка-остатък застава на позиция с по-голям индекс спрямо съответната си двойка-стена.

Algorithm 5: Destruction

Input: $H[1..n], L[1..n], k$ - arrays of naturals and a natural number

```
1  $A \leftarrow \text{malloc}(2n, \text{pair})$ 
2 for  $i \leftarrow 1$  to  $n$  do
3    $A[2 * i - 1] \leftarrow (H[i] \bmod L[i], H[i] \bmod L[i])$ 
4    $A[2 * i] \leftarrow (H[i] - (H[i] \bmod L[i]), L[i])$ 
5  $\text{CustomMergeSort}(A, 1, 2n)$ 
6  $\text{answer} \leftarrow 0$ 
7 for  $i \leftarrow 1$  to  $2n$  do
8    $t \leftarrow \min\left\{k, \left\lceil \frac{A[i].h}{A[i].l} \right\rceil\right\}$ 
9    $\text{answer} \leftarrow \text{answer} + t \times A[i].l$ 
10   $k \leftarrow k - t$ 
11 return  $\text{answer}$ 
```

Ред 1 заделя достатъчно памет за двойките, които ще разделяме. Тук променливите от тип pair имат полета с имена h и l точно както сме ги описали в условието – височина и чупливост. Редове 2 до 4 се грижат за отделянето на една двойка на две отделни, както е описано в абзаца преди псевдокода. На ред 5 CustomMergeSort е модификация на класическия MergeSort, която сравнява двойките спрямо l -полето им. Това е линейна наредба, защото l -полетата са естествени числа, а $\mathbb{N}$ има линейна наредба относно релацията $<$. Редове 7 до 10 обработват двойките в напълно идентичен начин като в решението на FRACTIONALKNAPSACK.

Решението на задачата е композиция на линейни обхождания и сортиране чрез MergeSort, съответно алгоритъмът завършва и времевата му сложност е $\mathcal{O}(n \log n)$.

Оценяване на задача 3 3 точки за коректен псевдокод, 5 за обоснована коректност (в зависимост от предложения псевдокод, не предложеното решение по-горе)¹ и 2 точка за времевата сложност² и завършването на алгоритъма.

Допълнително, съществува решение ползващо структура от данни “пирамида”, което не се възползва от разделяне на двойките на две части, но там съществена част от доказателството е да се покаже, че алгоритъмът завършва, когато пирамидата е празна или k стане равно на 0. Следва трите точки от наблюдението да се пренесат в завършването и времевата сложност на алгоритъма.

Решения с времева сложност $\mathcal{O}((n + k) \log n)$ да се оценяват с не повече от 5 точки.

Бонусът от 5 точки се получава, ако е предложено решение, което използва линеен алгоритъм за строене на пирамидата и допълнително обосновано завършва, когато всички войници са използвани (върти while-цикъла, докато $k \geq 0$).

¹На предложеното решение трябва да се обоснове наредбата на двойките-стени и двойките-остатъци, допълнително коректността на решението на FRACTIONALKNAPSACK.

²Ако се ползва while-цикъл до $k \geq 0$ или изпразване на опциите, това следва да се сложи тук.